Short Technical Note

FP16 versus BF16 in Deep Learning: Theory, Numerical Behavior, and Practical PyTorch Training

Your Name[a]

[a]*Department / Lab / University*

Draft for Overleaf Free — kept intentionally lightweight for fast compilation

Abstract

This document is a compact paper-style template for organizing a short study on FP16 versus BF16. The structure is designed for a five-page note with a clear research flow: background, theory and equations, architecture and experimental design, results, and conclusion. The template is intended for PyTorch-based experiments where the same task, same input-output definition, and same evaluation protocol are compared under full precision, FP16 mixed precision, and BF16 mixed precision.

Keywords: *FP16, BF16, mixed precision, PyTorch, numerical stability, throughput*

1. BACKGROUND

The goal of this note is to compare **FP16** and **BF16** under the same learning problem and the same model family. A good short-paper question is the following: *when the architecture, dataset, optimizer, and stopping rule are fixed, do FP16 and BF16 produce a measurable difference in throughput, memory usage, and training stability?*

For a compact yet meaningful study, a transformer encoder is a strong default choice because the workload is dominated by matrix multiplications. If you prefer a simpler vision baseline, **ResNet-18** on CIFAR-10 is also suitable. In both cases, keep the task definition fixed and change only the precision mode.

2. THEORY AND MATHEMATICAL FORMULATION

A normalized floating-point number may be written as

$$x = (-1)^s \cdot 2^{e-b} \cdot (1+m), \tag{1}$$

where s is the sign bit, e is the exponent field, b is the exponent bias, and m is the fractional mantissa term.

The practical difference between FP16 and BF16 is the bit allocation:

$$\text{FP16} : (1 \text{ sign},\ 5 \text{ exponent},\ 10 \text{ fraction}), \tag{2}$$

$$\text{BF16} : (1 \text{ sign},\ 8 \text{ exponent},\ 7 \text{ fraction}). \tag{3}$$

Therefore, FP16 usually offers finer local resolution, whereas BF16 offers a much wider dynamic range. A useful summary is

$$\text{range robustness} \uparrow \Rightarrow \text{BF16 is often easier to train}, \tag{4}$$

while

$$\text{local quantization resolution} \uparrow \Rightarrow \text{FP16 is often finer within a narrower range}. \tag{5}$$

If the loss is denoted by $\mathcal{L}(\theta)$, then standard gradient descent uses

$$\theta_{t+1} = \theta_t - \eta\, \nabla_\theta \mathcal{L}(\theta_t). \tag{6}$$

Under FP16, very small gradients may underflow more easily, which motivates loss scaling:

$$\tilde{\mathcal{L}}(\theta) = S\, \mathcal{L}(\theta), \tag{7}$$

$$\nabla_\theta \mathcal{L}(\theta) = \frac{1}{S} \nabla_\theta \tilde{\mathcal{L}}(\theta), \tag{8}$$

where $S > 0$ is a scale factor. In a short paper, this is the central mathematical reason why BF16 and FP16 can behave differently even when their wall-clock speed looks similar.

3. ARCHITECTURE AND EXPERIMENTAL DESIGN

3.1. Recommended architecture

For a concise experiment, use one architecture only. A clean choice is a **6-layer transformer encoder** with fixed sequence length and fixed hidden size. This makes the comparison easier because the same attention blocks, MLP blocks, normalization layers, and optimizer states are used across all precision modes.

3.2. Controlled comparison

To keep the comparison valid, hold the following constant:

- dataset and train/validation split,
- model depth, width, and token length,
- optimizer and learning rate schedule,
- number of epochs or update steps,
- seed and data order when possible,
- evaluation metric and stopping rule.

The quantities to report are throughput, step time, peak memory, final validation score, and instability events such as NaN or Inf. A concise result table should have the form shown in Table 1.

Table 1. Suggested result table for the FP16 vs BF16 study.

Mode	ms/step	samples/s	peak GB	val metric
FP32	–	–	–	–
FP16	–	–	–	–
BF16	–	–	–	–

Replace the placeholders with measured values from the same hardware, same batch policy, and same model.

3.3. PyTorch training loop

Use `autocast` for mixed precision and keep the model weights in the standard PyTorch workflow. A compact example is shown in Code 1.

Correspondence: your.email@domain.edu

This slim version removes minted, bibliography backends, decorative environments, and sample assets.

```python
device = torch.device("cuda")
model = MyTransformer().to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)
criterion = torch.nn.CrossEntropyLoss()

amp_dtype = torch.bfloat16  # or torch.float16
scaler = torch.amp.GradScaler("cuda", enabled=(amp_dtype == torch.
    ↪ float16))

for x, y in loader:
    x = x.to(device, non_blocking=True)
    y = y.to(device, non_blocking=True)
    optimizer.zero_grad(set_to_none=True)

    with torch.autocast(device_type="cuda", dtype=amp_dtype):
        logits = model(x)
        loss = criterion(logits, y)

    if amp_dtype == torch.float16:
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
    else:
        loss.backward()
        optimizer.step()
```

Code 1. Minimal PyTorch mixed-precision loop.

4. EXPERIMENTAL RESULTS

This section should report measured outcomes, not expectations only. A concise narrative can follow the structure below:

- **Speed**: BF16 and FP16 throughput relative to FP32.
- **Memory**: peak allocated memory under the same batch size.
- **Stability**: whether training diverged, produced NaNs, or required extra tuning.
- **Accuracy**: whether final validation performance changed meaningfully.

A useful interpretation pattern is: if BF16 is more stable but speed is similar, then dynamic range is likely the dominant factor; if FP16 is faster but less stable, then the experiment is exposing a trade-off between raw tensor-core efficiency and numerical robustness.

5. CONCLUSION

The final conclusion should answer one question only: *for this exact architecture, dataset, and hardware, which precision mode gives the best practical trade-off?* A strong conclusion usually contains three statements:

1. the theoretical reason for the observed difference,
2. the measured performance difference,
3. the recommended default precision for future experiments.

PRACTICAL NOTES FOR EXTENDING THIS NOTE

- If you later need a longer paper, add a dedicated related-work section.
- If you later need many references, switch from `thebibliography` to a `.bib` workflow.
- If you later need large code blocks, keep `listings` unless syntax highlighting quality becomes critical.

REFERENCES

[1] Paulius Micikevicius et al., *Mixed Precision Training*, International Conference on Learning Representations, 2018.

[2] Shibo Wang and Pankaj Kanwar, *BFloat16: The Secret to High Performance on Cloud TPUs*, Google Cloud Blog, 2019.

[3] PyTorch Documentation, *Automatic Mixed Precision Package – torch.amp*.